

Brac University

Department of Electrical & Electronic Engineering

Summer 2025

Course Number : EEE103L

Course Title : Computer Programming Laboratory

Section :06

Group No : 03



Lab Report

Experiment no:

Name of the experiment:

Introduction to C programming, I/O, Datatypes

Prepared by:

Name: Souvik Barman Ratul ID: 24121205

All Group members:

<i>Sl.</i>	<i>ID</i>	<i>Name</i>
1.	24321069	Tahmid Jamil
2.	24121125	Shakila Hoque Nanjiba
3.	24121283	Sumaiya Fahmida Zerin
4.	24121205	Souvik Barman Ratul
5.	24121346	Mohammad Sadat Hossain Bhuiyan

Group contribution:

Writing : Souvik Barman Ratul

Coding : Shakila Hoque Nanjiba

Testing : Mohammad Sadat Hossain Bhuiyan

Problem Solving : Sumaiya Fahmida Zerin

Discussion : Tahmid Jamil

But each member of group 3 contributed to each segment by working together collaboratively.

Title and Objective:

- Title: Introduction to C programming, I/O, Datatypes
- Objective: To learn basic C programming concepts including input/output operations, data types, and arithmetic operations through practical coding exercises.

Theoretical background:

This experiment introduces fundamental programming concepts in the C language.

- printf() for output .
- Data types such as int, float, double, and char.
- Type casting and format specifiers (e.g., %d, %f)
- Arithmetic and modulus operations
- Writing basic C programs in an IDE (CodeBlocks) for console-based output.

Tasks & Our Solutions:

Task 1: First find the value of each of the following arithmetic expressions in C. Then step by step, show in detail, how to hand calculate those values? Simply, you will write your investigation report on how C in CodeBlocks found those answers.

For example, if you are told to write $5+4*3/2-1$, Your answer should be like: $= 5+4*3/2-1 = 5+12/2-1 = 5+6-1 = 11-1 = 10$, in the investigation report

- A) $3+4.8*2$
- B) $12\%16$
- C) $2.*6/5$
- D) $5.2+12/8$
- E) $4 -(6+18.0/3)/3$
- F) $17-12*4$
- G) $2*6/5.$
- H) $7/(4*2)$
- I) $2*6/5$
- J) $19\%13$
- K) $36/60$

ANSWER:

Code:

```
A)  $3+4.8*2$   
#include<stdio.h>  
  
int main(){  
float x=3+4.8*2;  
printf("%f",x);  
return 0;  
}
```

```
1  
2 #include<stdio.h>  
3 int main(){  
4 float x=3+4.8*2;  
5 printf("%f",x);  
6 return 0;  
7 }  
8
```

Output
12.600000
=== Code Execution Successful ===

B) $12\%16$

```
#include<stdio.h>

int main(){

float x=12%16;

printf("%f",x);

return 0;

}
```

```
1
2  #include<stdio.h>
3  int main(){
4  float x=12%16;
5  printf("%f",x);
6  return 0;
7  }
8
```

Output

12.000000

=== Code Execution Successful ===

C) $2.*6/5$

```
#include<stdio.h>

int main(){

float x=2.*6/5;

printf("%f",x);

return 0;

}
```

```
1
2  #include<stdio.h>
3  int main(){
4  float x=2.*6/5;
5  printf("%f",x);
6  return 0;
7  }
```

Output

2.400000

=== Code Execution Successful ===

D) $5.2+12/8$

```
#include<stdio.h>

int main(){

float x=5.2+12/8;

printf("%f",x);

return 0;

}
```

```
main.c .C
1
2  #include<stdio.h>
3  int main(){
4  float x=5.2+12/8;
5  printf("%f",x);
6  return 0;
7  }
```

Output

6.200000

=== Code Execution Successful ===

E) $4 -(6+18.0/3)/3$

```
#include<stdio.h>

int main(){

float x=4 -(6+18.0/3)/3;

printf("%f",x);

return 0;

}
```

```
main.c
2  #include<stdio.h>
3  int main(){
4  float x=4 -(6+18.0/3)/3;
5  printf("%f",x);
6  return 0;
7  }
```

Output

0.000000

=== Code Execution Successful ===

F) $17-12*4$

```
#include<stdio.h>

int main(){

float x=17-12*4;

printf("%f",x);

return 0;

}
```

```
1
2 #include<stdio.h>
3 int main(){
4 float x=17-12*4;
5 printf("%f",x);
6 return 0;
7 }
```

Output

-31.000000

=== Code Execution Successful ===

G) $2*6/5$.

```
#include<stdio.h>

int main(){

float x=2*6/5.;

printf("%f",x);

return 0;

}
```

```
1 #include<stdio.h>
2 int main(){
3 float x=2*6/5.;
4 printf("%f",x);
5 return 0;
6 }
```

Output

2.400000

=== Code Execution Successful ===

H) $7/(4*2)$

```
#include<stdio.h>

int main(){

float x=7/(4*2);

printf("%f",x);

return 0;

}
```

```
main.c
1 #include<stdio.h>
2 int main(){
3 float x=7/(4*2);
4 printf("%f",x);
5 return 0;
6 }
```

Output

0.000000

=== Code Execution Successful ===

I) $2*6/5$

```
#include<stdio.h>

int main(){

float x=2*6/5;

printf("%f",x);

return 0;

}
```

```
1 #include<stdio.h>
2 int main(){
3 float x=2*6/5;
4 printf("%f",x);
5 return 0;
6 }
```

Output

2.000000

=== Code Execution Successful ===

J) $19\%13$

```
#include<stdio.h>

int main(){
float x= 19%13;
printf("%f",x);
return 0;
}
```

Code	Output
<pre>1 #include<stdio.h> 2 int main(){ 3 float x= 19%13; 4 printf("%f",x); 5 return 0; 6 }</pre>	<pre>6.000000 === Code Execution Successful ===</pre>

K) $36/60$

```
#include<stdio.h>

int main(){
float x= 36/60;
printf("%f",x);
return 0;
}
```

Code	Output
<pre>1 #include<stdio.h> 2 int main(){ 3 float x= 36/60; 4 printf("%f",x); 5 return 0; 6 }</pre>	<pre>0.000000 === Code Execution Successful ===</pre>

Handcalculation:

A. $3+4.8*2$

$$= 3+9.6$$

$$=12.6$$

B. $12\%16$

$$=12 \quad (\% \text{ is the modulus operator for integers only})$$

C) $2.*6/5$

$$= 2.0*6/5$$

$$= 12.0/5$$

$$= 2.4 \text{ (float division)}$$

D) $5.2+12/8$

$$=5.2+1 \text{ (because both are integer)}$$

$$=6.2$$

E) $4 - (6 + 18.0/3)/3$

$= 4 - (6 + 6.0)/3$

$= 4 - 12.0/3$

$= 4 - 4.0$

$= 0$

F) $17 - 12 * 4$

$= 17 - 48$

$= -31$

G) $2 * 6/5.$

$= 12/5.0$

$= 2.4$

H) $7/(4 * 2)$

$= 7/8$

$= 0$

I) $2 * 6/5$

$= 12/5$

$= 2.4$ (integer division)

J) $19 \% 13$

$= 6$

K) $36/60$

$= 0$

Task 2: Try the following programs and note what happens. Then try to explain the behaviors line by line.

```
int main()
{
    char b;
    int i = 257;
    double d = 323.142;
    b = i;
    printf("%c\n", b);
    b = (char) i;
    printf("%c\n", b);
    i = (int) d;
    printf("%d\n", i);
    b = (char) d;
    printf("%c\n", b);
}
```

ANSWER:

Here, this program explains type conversion between int, char and double. Firstly, an integer value 257 is stored into a char variable. A char can only hold 1 byte, it keeps only the last part of the number, which ends up being 1. Printing this as a character gives a non-printable character on the screen.

In the second part, this program does the same thing but in this time the number converts into a character. This still keeps 1 byte and the outcome is 1 which still prints non-printable characters on the screen.

In the third part, this program converts the double variable into an integer. Here 323.142 turns into 323 because it basically removes the decimal digits. Prints the 323 number on the screen.

Finally, the program converts that double value to a char. It first becomes the integer 323 and then since a char can only store 1 byte. This results in the number 67, which means the character 'C' in the ASCII table. So, printing this shows the letter 'C' on the screen.

Overall we can see how type conversion works in C programming.

Task 3: Here is a pseudo program that computes the number of miles that light will travel in a specified number of days. Try to compile and run the following C program.

If it does not work, make necessary corrections so that it compiles, runs and gives correct output.

```
Lightspeed = 186000
days = 1000;      //specify number of days here
seconds = Days*24*60*60; /*convert to seconds/
distance = lightspeed * Second;    //compute distance
printf("In " + days + " days light will travel about ");
printf(Distance + "miles.");
```


ANSWER:

Code:

```
#include <stdio.h>

int main() {

    long lightspeed = 186000;

    int days = 1000;

    long seconds = days*24*60*60;

    long distance= lightspeed*seconds;

    printf("In %d days, light will travel about %ld miles.\n", days, distance);

    return 0;

}
```



```
main.c .C
1
2 #include <stdio.h>
3 int main() {
4     long lightspeed = 186000;
5     int days = 1000;
6     long seconds = days*24*60*60;
7     long distance= lightspeed*seconds;
8     printf("In %d days, light will travel about %ld miles.\n", days, distance);
9     return 0;
10 }
```

Output

In 1000 days, light will travel about 16070400000000 miles.

=== Code Execution Successful ===

Task 4: Write a program in C that can calculate the area of a circle up to 4 decimal points taking the radius as input. Take the value of radius from the user in the console window and radius can be any integer or decimal fraction number. Print the value of the area.

(Submit your code in report by copy pasting it and also attach a screenshot of your result in command window)

ANSWER:

Code:

```
#include <stdio.h>

int main() {

    double r, area;

    printf("Enter the radius of the circle:\n");

    scanf("%lf", &r);

    area = 3.1416 * r * r;

    printf("The area of the circle is: %.4lf\n", area);

    return 0;

}
```

main.c

```
1
2 #include <stdio.h>
3 int main() {
4     double r, area;
5     printf("Enter the radius of the circle:\n");
6     scanf("%lf", &r);
7     area = 3.1416 * r * r;
8     printf("The area of the circle is: %.4lf\n", area);
9     return 0;
10 }
```

Output

```
Enter the radius of the circle:
103
The area of the circle is: 33329.2344
```

=== Code Execution Successful ===

Discussion:

In this lab, we learned how basic C programming works, especially with input/output, data types, and simple math operations. At first, we struggled a bit with using the right data types and format specifiers, but we figured it out by testing, correcting mistakes, and discussing with team members. Overall, it was a good way to understand how C actually works behind the scenes.